Lab 10 – Code Review and Quality: Using AI to Improve Code Quality and Readability

Name: Syed Murtaza

Hall ticket no: 2303A51259

Batch no: 19

Task 1: Variable Naming Issues

Prompt:

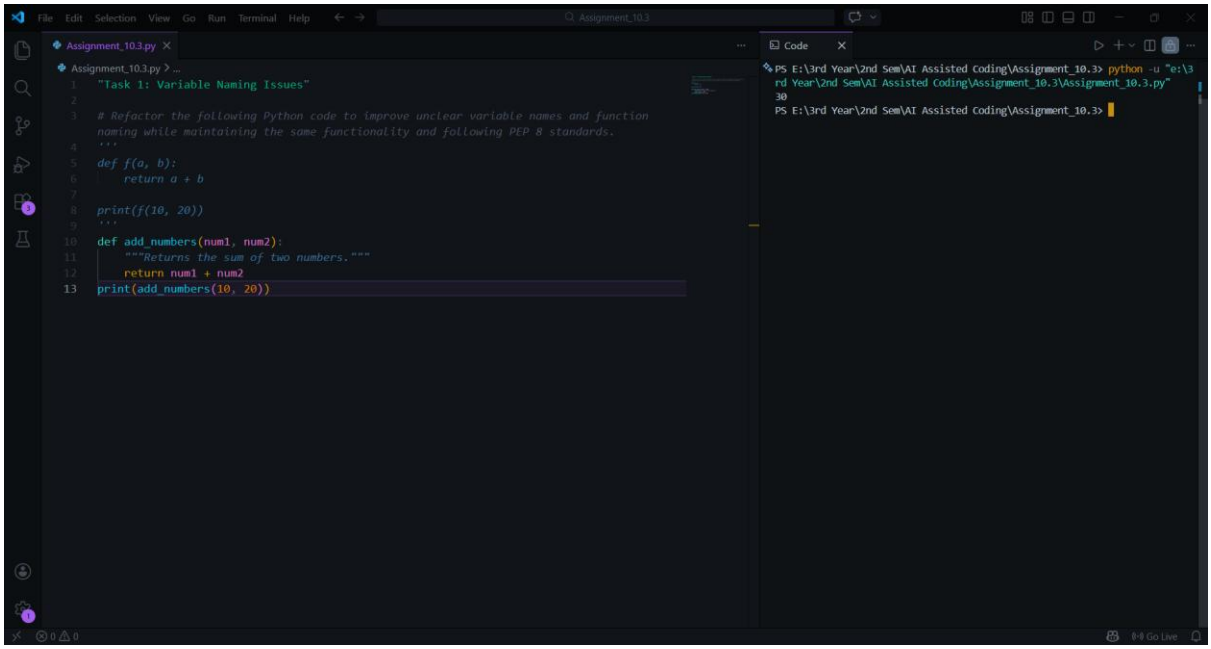
Refactor the following Python code to improve unclear variable names and function naming while maintaining the same functionality and following PEP 8 standards.

def f(a, b):

return a + b

print(f(10, 20))

Code & Output:



Explanation:

The original code uses unclear names such as *f*, *a*, and *b*, which do not describe their purpose. The AI-refactored version replaces them with meaningful identifiers like

add_numbers, *first_number*, and *second_number*. This improves readability and makes the function's purpose immediately clear. The new version also follows PEP 8 naming conventions, enhancing maintainability without changing functionality.

Task 2: Missing Error Handling

Prompt:

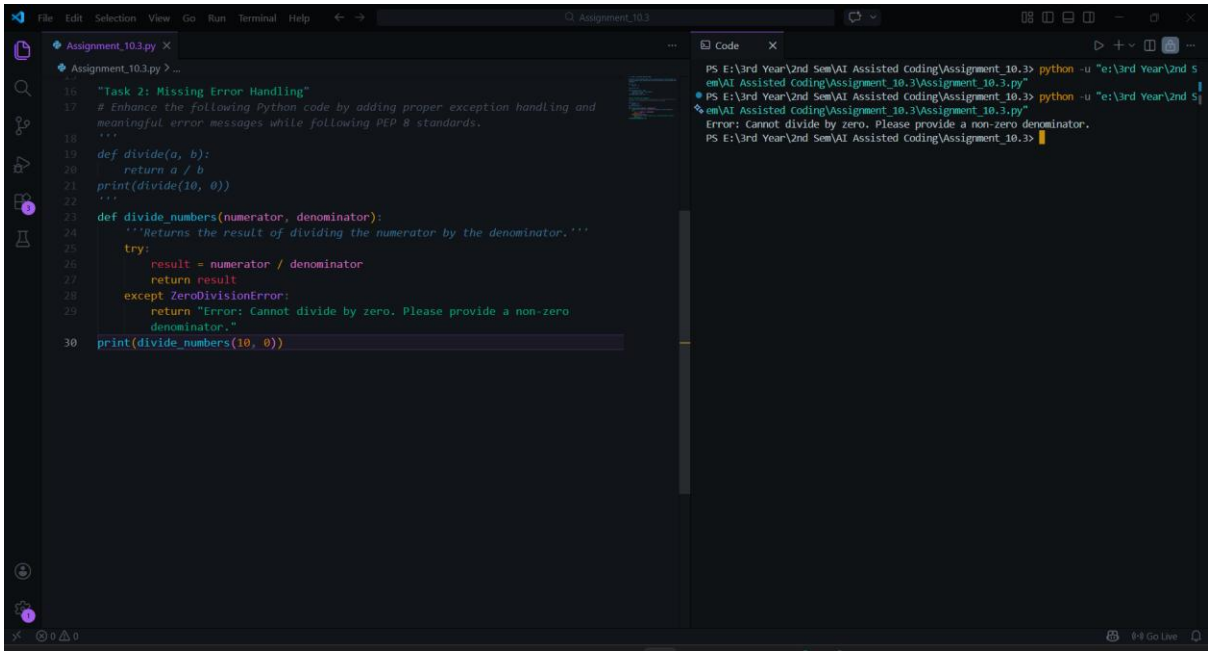
Enhance the following Python code by adding proper exception handling and meaningful error messages while following PEP 8 standards.

def divide(a, b):

return a / b

print(divide(10, 0))

Code & Output:



Explanation:

The original code does not handle division by zero, which causes a runtime error. The AI-enhanced version introduces a try-except block to handle this scenario gracefully. Meaningful variable names improve clarity. This improves robustness, prevents crashes, and ensures better user experience. Error handling is essential for writing production-quality software.

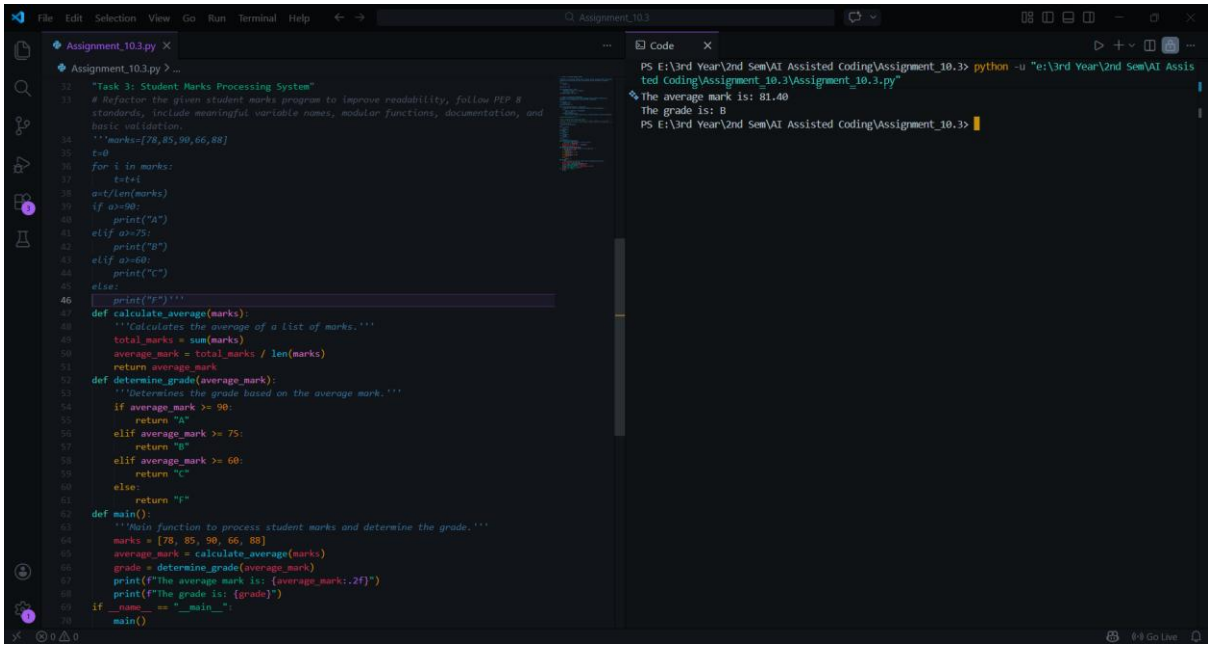
Task 3: Student Marks Processing System

Prompt:

Refactor the given student marks program to improve readability, follow PEP 8 standards, include meaningful variable names, modular functions, documentation, and basic validation.

```
marks=[78,85,90,66,88]
t=0
for i in marks:
    t=t+i
a=t/len(marks)
if a>=90:
    print("A")
elif a>=75:
    print("B")
elif a>=60:
    print("C")
else:
    print("F")
```

Code & Output:



Explanation:

The original code lacks structure, meaningful variable names, and documentation. The AI-refactored version modularizes the logic into a function, improves variable naming, and follows PEP 8 formatting. The use of built-in sum() improves efficiency and readability. This version is more maintainable and reusable.

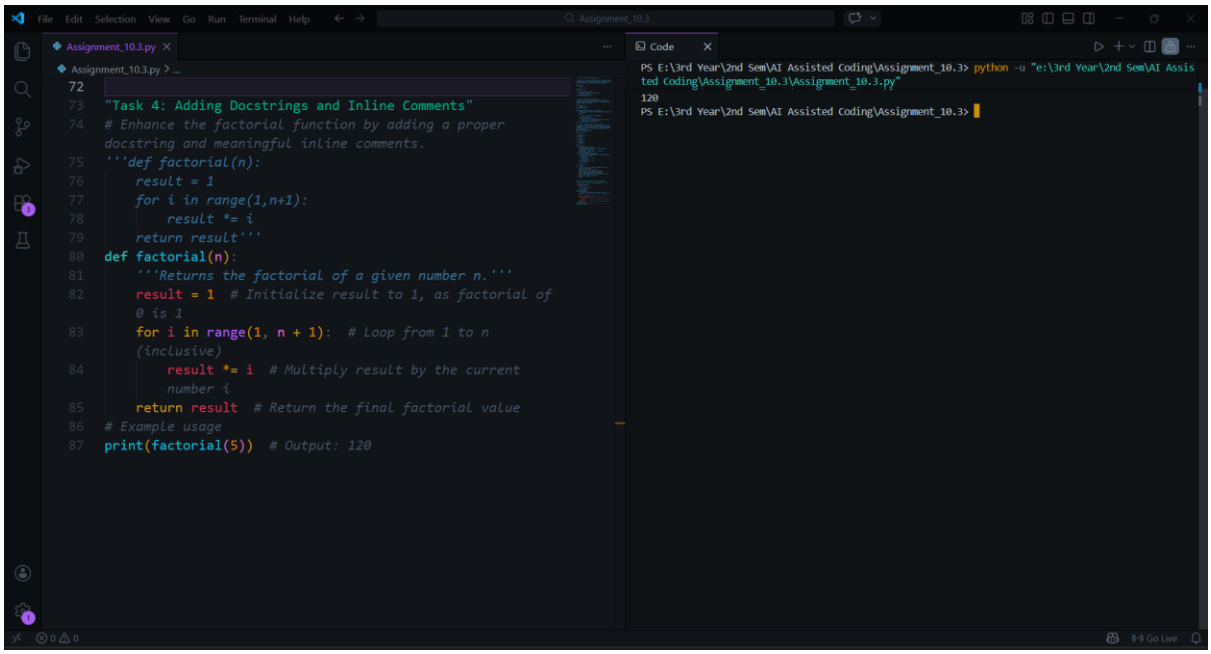
Task 4: Adding Docstrings and Inline Comments

Prompt:

Enhance the factorial function by adding a proper docstring and meaningful inline comments.

def factorial(n):
result = 1
for i in range(1,n+1):
*result *= i*
return result

Code & Output:



Explanation:

The AI-enhanced function adds a docstring describing the function's purpose and parameter. Inline comments explain each logical step. Variable naming is improved for clarity. This enhances readability and documentation quality.

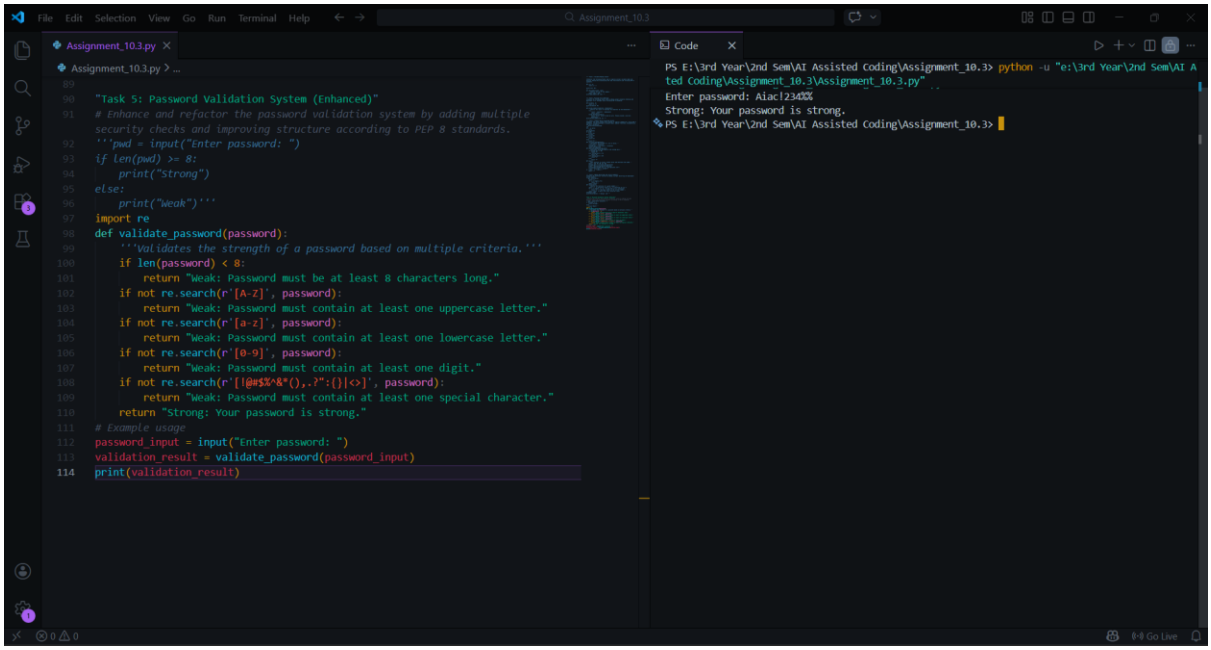
Task 5: Password Validation System (Enhanced)

Prompt:

Enhance and refactor the password validation system by adding multiple security checks and improving structure according to PEP 8 standards.

pwd = input("Enter password: ")
if len(pwd) >= 8:
print("Strong")
else:
print("Weak")

Code & Output:



Explanation:

The enhanced program introduces multiple password security rules including uppercase, lowercase, digits, and special characters. The logic is modularized into a function with documentation. Compared to the original version, the new program is more secure, readable, and maintainable.

Final Conclusion:

This lab demonstrated how AI-assisted coding tools can be effectively used for automated code review and quality enhancement. AI suggestions helped improve variable naming, added proper error handling, enhanced documentation, strengthened password security, and ensured compliance with PEP 8 coding standards. The refactored programs became more readable, maintainable, and robust compared to their original versions. However, human evaluation remains essential to validate AI-generated improvements and ensure they align with software engineering best practices.